Anonymous Complaint Management System (ACMS): A Schema-Level Anonymous, Role-Aware Web Application Built on the MERN Stack

[1]Karan Jagdishrao Keche, [2]Vaishnavi Sanjay Kathar, [3]Vitthal Narhari Kendre, [4]Somashekhar Arabali

[1,2,3,4]Department of Computer Engineering, Sinhgad Institute of Technology and Science (SITS), Narhe, Pune – 411041, Maharashtra, India | Affiliated to Savitribai Phule Pune University (SPPU)

[1]karankeche.sits.comp@gmail.com | [2]vaishnavikathar.sits.comp@gmail.com | [3]vitthalkendre.sits.comp@gmail.com | [4]somashekhararabali.sits.comp@gmail.com

Abstract—Students in academic institutions frequently hesitate to raise legitimate grievances due to fear of identification or retaliation, and existing complaint portals rarely offer a technical, rather than merely procedural, guarantee of anonymity. This paper presents the Anonymous Complaint Management System (ACMS), a full-stack web application built on the MERN stack (MongoDB, Express.js, React.js, Node.js) that enforces anonymity at the data-model level by omitting any submitter reference from a complaint document when the student chooses to submit anonymously, ensuring no personal identifier is ever persisted alongside an anonymous complaint. ACMS is implemented as a React.js single-page application that communicates with a custom Node.js and Express.js REST API over HTTPS; the API in turn reads and writes to a MongoDB database through Mongoose schemas, using MongoDB Atlas for the production deployment and MongoDB Compass for local development and testing. Authentication is implemented with JSON Web Tokens (JWT) issued and verified by the Express server, and passwords are hashed using bcrypt before being stored. Role-based access control is enforced through Express middleware that inspects the JWT payload on every protected route, distinguishing student and administrator privileges. Beyond the system description, this paper formalises the anonymity guarantee as a data-layer property, states its precise threat-model boundaries, and reports functional and security verification performed directly against the live deployment and its underlying MongoDB documents, together with an explicit statement of the evaluation's scope and limitations — including the absence of load testing and independent security audit — so that the anonymity claim is neither overstated nor mistaken for a guarantee at the network or infrastructure layer. The application supports anonymous and named complaint submission, a structured complaint lifecycle, a role-aware dashboard for students and administrators, and an administrator reply mechanism visible to the submitting student. The system is styled with Tailwind CSS and deployed live on Vercel, with the API hosted as a separate Node.js service.

Index Terms—Anonymous Complaint System, MERN Stack, React.js, Node.js, Express.js, MongoDB, Mongoose, JWT, bcrypt, Role-Based Access Control, Grievance Redressal, Vercel, Academic Institutions

I. INTRODUCTION

In any academic institution, the availability of a reliable, impartial, and confidential grievance channel is a cornerstone of institutional governance. Students frequently hesitate to raise complaints — particularly those concerning faculty conduct, peer harassment, examination irregularities, or administrative misconduct — out of genuine and well-founded fear of identification and retaliation [2].

Traditional complaint mechanisms fail at the technical level: submitted documents are associated with login credentials, email metadata, or digital signatures that can inadvertently reveal the submitter's identity even when institutional policy promises confidentiality. The **Anonymous Complaint Management System (ACMS)** addresses this gap through deliberate, data-model-level design: when a student chooses to submit a complaint anonymously, the backend controller simply omits the submitter reference from the document written to MongoDB, so that no identifying field is ever associated with that complaint at any layer.

ACMS is built on the MERN stack: a React.js single-page application for the frontend, a Node.js and Express.js REST API as the application's business-logic layer, and MongoDB — accessed through Mongoose — as the data layer. This is a deliberately conventional three-tier web architecture, chosen so that the project team retains full control over validation, authentication, and access-control logic rather than delegating it to a third-party platform. Authentication and authorisation are implemented entirely in the Express layer using JWT and bcrypt, with role checks performed in custom middleware on every protected route.

Key Contributions of this Paper:

- A data-model-level anonymity mechanism that omits the submitter reference for anonymous complaints, implemented and verified in the live MERN application
- A formal statement of the anonymity guarantee as a data-layer property, together with an explicit threat-model boundary distinguishing it from network- or infrastructure-layer anonymity
- A complete description of a custom Express.js REST API with JWT authentication and role-based middleware for an academic CMS
- A structured complaint lifecycle with auditable status transitions stored in MongoDB
- A role-aware single-page application serving both student and administrator dashboards from one codebase
- A discussion of the trade-offs of a self-managed MERN backend versus a managed Backend-as-a-Service for a resource-constrained student project
- Functional and data-layer verification of the deployed system at complaintmanagementsystem-eight.vercel.app, together with an explicit statement of the evaluation's scope and limitations

Paper Organisation: Section II surveys related work. Section III analyses the problem and existing systems. Section IV describes the development and evaluation methodology. Section V presents the system architecture and database schema design. Section VI formalises the anonymity guarantee and its threat-model boundaries. Section VII details the implementation. Section VIII reports functional and security test results. Section IX discusses these results against the paper's stated goals. Section X states the evaluation's limitations explicitly. Section XI concludes and outlines future work.

II. LITERATURE SURVEY

A. Microservice-Based Complaint Systems

Suryotrisongko et al. [1] proposed a backend architecture for public complaint systems using a microservice-based Spring Boot framework, published in Procedia Computer Science (Elsevier, ISICO 2017). Their work demonstrated that decoupled microservice design improves scalability and maintainability of complaint platforms. ACMS adopts a lighter single-service Express.js API rather than a microservice decomposition, which is appropriate given the smaller scope of a single-

institution academic deployment, though the underlying motivation — separating business logic from the data store behind a well-defined API — is shared with their approach.

B. E-Governance and Institutional Grievance Portals

India's CPGRAMS mandates user registration linked to PAN or Aadhaar, making anonymity structurally impossible. Joshi et al. [2] observed that participation rates in government grievance portals decline significantly when identity disclosure is mandatory. Kumar and Singh [3] examined twelve Indian university complaint portals and found that none implemented genuine anonymisation; all relied on administrative promises rather than a technical guarantee enforced in the data model itself.

C. Industrial Whistleblower Systems

Corporate platforms such as EthicsPoint (NAVEX Global) [4] implement anonymity via third-party report routing. While effective, such systems are proprietary, expensive, and unsuitable for academic workflows on a student project budget, motivating a self-hosted approach instead.

D. Anonymisation Techniques in Web Applications

Decoupling a record from its originating user by omitting or nulling a reference field is a well-established pattern in application data modelling. Date [5] identified this general approach as the "entity disassociation" pattern in relational systems, and the same principle applies directly to a document database: a Mongoose schema can simply leave the submitter field absent or null on an anonymous document. Mivule [6] explored token-based anonymisation in healthcare systems as an alternative; ACMS adopts the simpler omitted-reference approach since full unlinkability — not pseudonymity — is the goal for anonymous complaints.

E. Authentication and Security Standards

Jones et al. [7] defined JWT (RFC 7519) as a compact, digitally signed token format enabling stateless session management, which is exactly how ACMS's Express server issues and verifies sessions without server-side session storage. Provos and Mazières [8] introduced bcrypt as an adaptive password hashing function that remains computationally expensive as hardware improves, which ACMS uses with a fixed salt round count for all stored credentials. Ferraiolo et al.'s NIST RBAC model [9] is the conceptual basis for the two-role (student/admin) middleware check implemented on every protected Express route in ACMS.

F. MongoDB and Mongoose in Academic Web Applications

MongoDB's document model is well suited to complaint records whose shape can vary slightly by category and priority, avoiding the rigid column definitions of a relational schema. Mongoose provides schema-level validation on top of MongoDB's native flexibility, which ACMS relies on to enforce required fields, enumerated status values, and reference relationships between complaints, users, and replies despite MongoDB itself being schemaless. Atlas was used for the production database, with Compass used locally during development for direct collection inspection and debugging.

G. Modern Frontend Technology

React.js [10] popularised declarative, component-based UI development with a virtual DOM, well-suited to ACMS's role-aware dashboards. Tailwind CSS was used for utility-first responsive styling without a custom CSS framework, and Axios was used as the HTTP client for all communication with the Express API.

H. Research Gap

The literature on academic complaint systems frequently claims anonymity as a feature without specifying the exact mechanism by which it is enforced in the underlying data model, and rarely documents the actual REST API and middleware design alongside a working, deployed system. ACMS, and this paper, address that gap by documenting precisely where in the Express controller and MongoDB schema the anonymity guarantee is enforced, and by reporting

functional test results against the live deployment rather than a theoretical design.

III. SYSTEM ANALYSIS

A. Problem Statement

Existing academic complaint systems suffer from: (i) lack of genuine, technical anonymity; (ii) poor lifecycle management; (iii) absence of real-time administrative oversight; (iv) inconsistent security practices; and (v) no separation of privilege between student and administrator roles. ACMS addresses all five using a self-managed Express.js API that the project team controls end to end.

B. Feasibility Study

Technical: React.js, Node.js, Express.js, and MongoDB are mature, widely documented technologies taught as part of the team's coursework, making a custom MERN backend a realistic choice for a four-person student team within the project timeline.

Operational: The system targets students and college staff who already use modern web browsers daily; no native app installation is required.

Economic: MongoDB Atlas's free tier (M0 cluster) and Vercel's free hobby tier together cover the hosting cost of the frontend and database for the scope of a final-year project; the Express API was deployed at no cost on a free Node.js hosting tier.

C. Functional Requirements

Table I: Functional Requirements of ACMS

FR-ID	Requirement	Status
FR-01	Student self-registration with email and password, hashed via bcrypt	**Done**
FR-02	Login issuing a signed JWT from the Express server	**Done**
FR-03	Anonymous complaint submission (submitter reference omitted)	**Done**
FR-04	Named complaint submission (submitter reference set to user's id)	**Done**
FR-05	Auto-generated MongoDB ObjectId per complaint	**Done**
FR-06	Admin dashboard with complaint counts by status	**Done**
FR-07	Admin: full complaint detail view with category and priority	**Done**
FR-08	Admin: update complaint lifecycle status	**Done**
FR-09	Admin: post reply visible to submitting student	**Done**
FR-10	Student: filter complaints by category and status	**Done**
FR-11	JWT-protected routes reject requests without a valid token	**Done**
FR-12	Role middleware blocks student accounts from admin-only endpoints	**Done**

D. Existing vs. Proposed System

Table II: Comparison — Existing System vs. ACMS

Feature	Existing System	ACMS
Anonymity	Policy only	Submitter field omitted in document
Backend	Often absent or ad hoc	Custom Express.js REST API
Access control	Inconsistent or none	JWT + role middleware on every route
Password storage	Plaintext or weak hash	bcrypt adaptive hashing
Lifecycle tracking	Manual, ad hoc	Structured status field in MongoDB

Feature	Existing System	ACMS
Role separation	None or UI-only	Middleware-enforced on server
Accessibility	Physical/VPN	HTTPS web (global, via Vercel)
Hosting cost	Often paid server	Free tier (Atlas + Vercel)

IV. METHODOLOGY

This section describes the research and development methodology followed in building and evaluating ACMS. The work combines a software engineering methodology, for constructing the system itself, with an empirical evaluation methodology, for verifying that the anonymity and access-control guarantees actually hold in the deployed artifact rather than only in design documentation.

A. Development Process

The team followed an incremental, feature-driven development process across four broad phases:

1. **Requirements elicitation and schema design** — deriving the functional requirements in Table I and the three Mongoose schemas (User, Complaint, Reply) from the problem statement in Section III-A, with the anonymity requirement (FR-03) treated as a data-modelling constraint from the outset rather than an access-control rule applied after the fact.
2. **Backend-first implementation** — building the Express.js REST API, JWT authentication, bcrypt password hashing, and role middleware before frontend integration, so that the anonymity and access-control guarantees could be verified directly against the API and database, independent of the UI.
3. **Frontend integration** — building the React.js single-page application against the already-verified API, with role-aware rendering driven by the JWT payload returned at login.
4. **Deployment and verification** — deploying the frontend to Vercel and the database to MongoDB Atlas, followed by the functional and security verification reported in Section VIII.

B. Research Approach for Anonymity Verification

Unlike conventional feature testing, verifying an anonymity guarantee requires evidence about what is absent from a system, not only what is present. The methodology adopted here is **constructive verification at the data layer**: rather than relying on code review alone to argue that the submitter is never recorded, the team inspected the actual MongoDB documents produced by the live controller logic using MongoDB Compass, confirming the literal absence of the submittedBy key on anonymous documents (Section VIII-C). This is a stronger evidentiary standard than asserting anonymity from source code alone, since it additionally rules out the possibility of a default value, a null placeholder, or a hidden field being written by Mongoose's schema defaults.

C. Evaluation Methodology

The system was evaluated along three dimensions, each with a distinct method appropriate to what is being measured:

- **Functional correctness** — black-box manual test cases (Table V) executed against the deployed frontend and API, covering registration, authentication, anonymous and named submission, role-based authorization, and the administrator workflow.
- **Security property verification** — direct inspection of stored data and HTTP responses to confirm that passwords are never stored in plaintext, that protected routes reject missing or invalid tokens, and that the anonymity field is genuinely absent rather than merely empty.
- **Architectural and design evaluation** — qualitative comparison against the existing-system baseline in Table II and against the related work in Section II, assessing the design on the criteria those sources themselves use to evaluate complaint and grievance systems (technical anonymity guarantee, access-control rigor, and auditability).

This paper does not report server load or response-time benchmarking under synthetic

traffic; that scope and the reasoning behind excluding it are stated explicitly in Section X (Limitations), consistent with the principle that an empirical claim should not appear in a paper unless it was actually measured on the system being described.

V. SYSTEM DESIGN

A. Architecture Overview

ACMS follows the conventional three-tier MERN architecture shown in Fig. 1. The React.js single-page application — styled with Tailwind CSS and using Axios for HTTP communication — is hosted on Vercel and calls a Node.js and Express.js REST API over HTTPS. The Express API exposes route handlers for authentication and complaint management, applies a JWT verification middleware and a role-checking middleware on protected routes, and communicates with MongoDB through Mongoose schemas. MongoDB Atlas hosts the production database; MongoDB Compass was used during development to inspect collections and verify document structure directly. A security layer spanning the Express tier applies bcrypt password hashing, JWT issuance and verification, HTTPS transport, and role-based route guards.

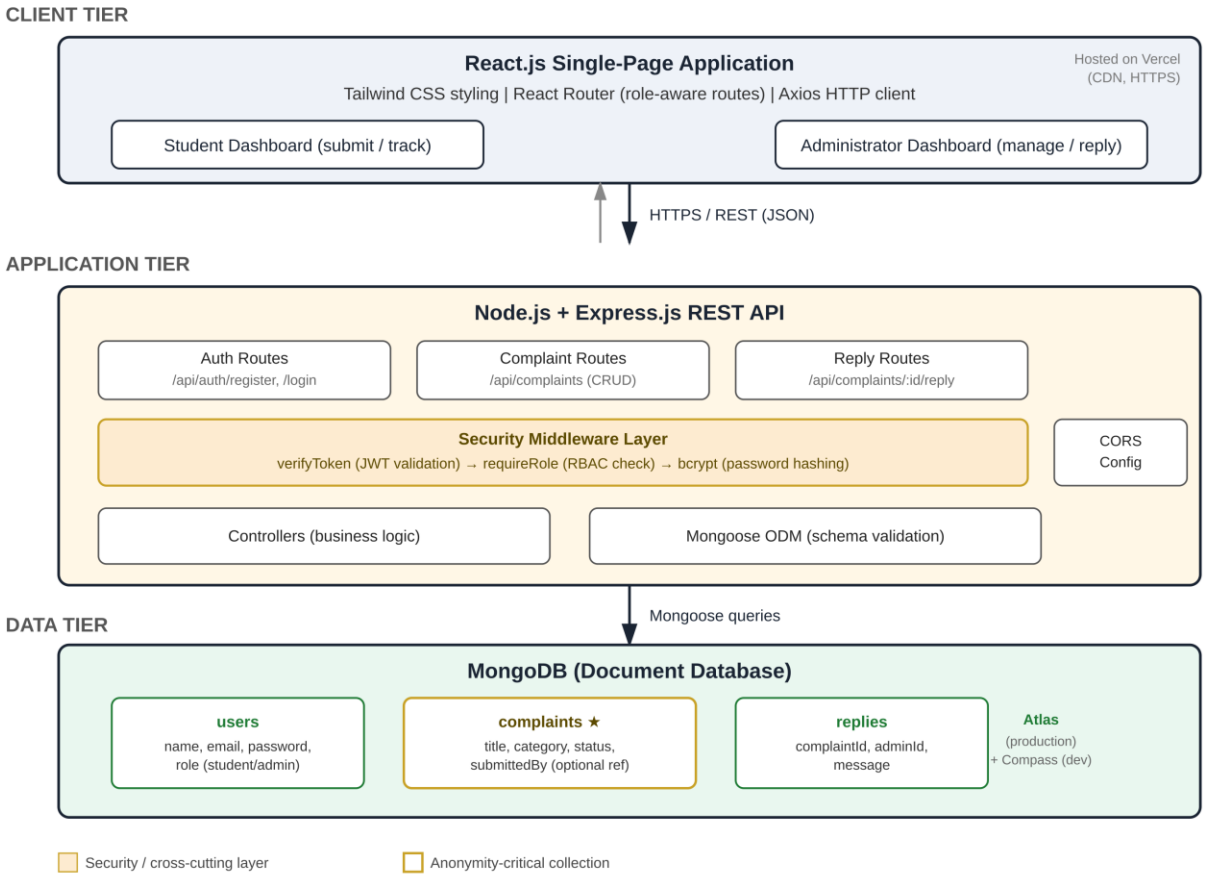

Fig. 1: ACMS Three-Tier MERN Architecture

B. Database Schema Design

Three Mongoose schemas constitute the ACMS data model. The anonymity mechanism resides in the **complaints.submittedBy** field, which is simply left out of the document when a student submits anonymously, rather than being populated with a reference to their user document.

Table III(a): Schema — User Collection

Field	Type	Constraint	Description
_id	ObjectId	PRIMARY KEY	Auto-generated by MongoDB
name	String	required: true	Display name
email	String	required, unique	Login email

Field	Type	Constraint	Description
password	String	required: true	bcrypt hash, never plaintext
role	String (enum)	required: true	'student' or 'admin'
createdAt	Date	default: Date.now	Registration timestamp

Table III(b): Schema — Complaint Collection

Field	Type	Constraint	Description
submittedBy	**ObjectId (ref: User)**	**NOT required ★**	**Omitted = Anonymous submission**
title	String	required: true	Short complaint title
description	String	required: true	Full complaint description
category	String	required: true	Academic / Infrastructure / etc.
priority	String (enum)	default: 'Low'	Low / Medium / High / Critical
status	String (enum)	default: 'Open'	Open / Pending / In Progress / Resolved / Rejected
createdAt	Date	default: Date.now	Submission timestamp

Table III(c): Schema — Reply Collection

Field	Type	Constraint	Description
_id	ObjectId	PRIMARY KEY	Auto-generated reply id
complaintId	ObjectId (ref: Complaint)	required: true	Parent complaint reference
adminId	ObjectId (ref: User)	required: true	Replying administrator
message	String	required: true	Reply content
createdAt	Date	default: Date.now	Reply timestamp

★ *Key anonymity mechanism: submittedBy is an optional reference field. The Express controller simply does not set it when isAnonymous is true, so no identifier is ever written to the document for an anonymous complaint.*

C. Anonymous Submission Data Flow

Fig. 2 traces what happens when a student submits a complaint with the anonymity toggle enabled. The React form sends a POST request to the Express API; the controller checks the isAnonymous flag and, if true, omits submittedBy entirely before calling Mongoose's .create() method. MongoDB stores the resulting document with no submitter field present. When an administrator later requests the complaint list, the same JWT-and-role-middleware-protected route returns all complaints, and the React admin dashboard renders any document missing submittedBy as “Anonymous.”

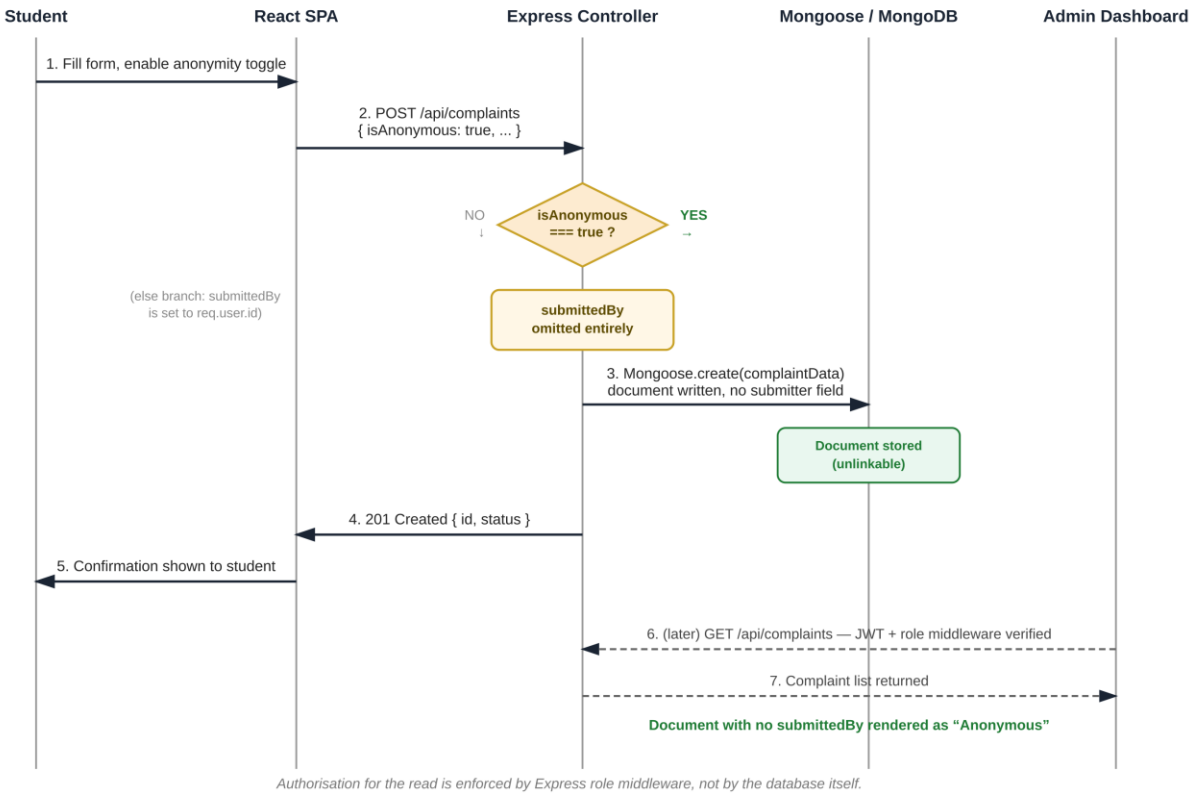

Fig. 2: Anonymous Complaint Submission Data Flow

D. Complaint Lifecycle State Machine

Every complaint transitions through a five-state lifecycle. A new complaint begins as Open. An administrator may move it to Pending, then to In Progress, and finally to Resolved. A complaint may also be moved directly from Open to Rejected, or from Pending to Rejected, if found invalid or unsupported by sufficient evidence. Each transition updates the status field and the Mongoose updatedAt timestamp, producing a complete audit trail regardless of whether the complaint was submitted anonymously.

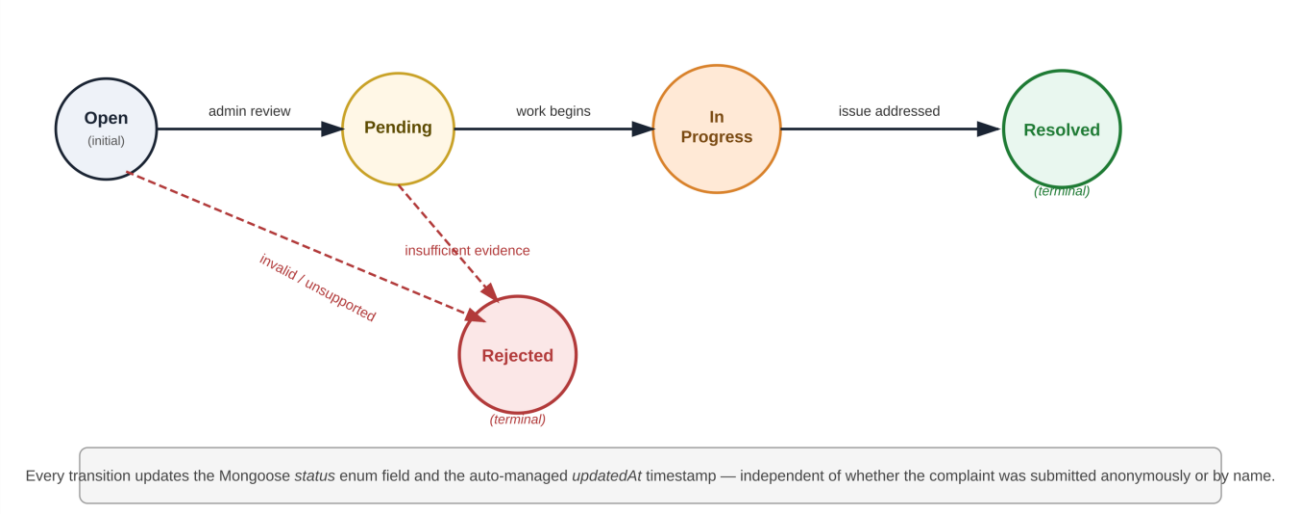

Fig. 3: Complaint Status Lifecycle State Machine

E. Role-Aware Routing

A single React Router configuration serves both student and administrator views from one codebase. On login, the JWT payload returned by the Express server includes the user's role, which the frontend uses to conditionally render either the student dashboard or the administrator dashboard. This client-side routing is a usability convenience only — it is the Express role

middleware, evaluated independently on every API request, that actually prevents a student account from reading another student's named complaints or from calling admin-only endpoints, even if the React routing were somehow bypassed.

VI. FORMAL MODEL OF THE ANONYMITY GUARANTEE

Section V described the anonymity mechanism procedurally, in terms of the Express controller and the Mongoose schema. This section restates the same mechanism as a small formal model, so that the precise scope and limits of the anonymity guarantee — what it does and does not protect against — can be stated unambiguously rather than left to informal description.

A. System and Data Model

Definition 1 (Complaint document). Let a complaint document be a tuple d = (id, t, c, p, s, u), where id is the MongoDB ObjectId, t is the title, c is the category, p is the priority, s is the lifecycle status, and $u \in U \cup \{\perp\}$ is the submitter reference: either a valid User ObjectId in the set U of registered users, or the distinguished absent value ⊥, meaning no submittedBy key is present on the underlying BSON document at all.

Definition 2 (Submission function). Let create: (fields, isAnonymous, requester) → d be the controller function exports.createComplaint. Its behaviour with respect to u is:

$$u = \perp \text{ if } isAnonymous = true, \quad u = requester.id \text{ if } isAnonymous = false$$

This is precisely the conditional in the implementation shown in Section V-B: the field is omitted, not set to null, zero, or any other sentinel value, when isAnonymous is true.

B. The Anonymity Property

Property 1 (Unlinkability at the data layer). For any complaint document d with u = ⊥, there exists no query Q expressible over the Complaint collection alone that returns a non-⊥ value for d.submittedBy, because the key is absent from the stored BSON document rather than present with an empty or null value. Formally, for all d with u = ⊥: d.submittedBy is undefined, not d.submittedBy = null.

This distinction matters because a null or empty-string placeholder would still occupy a queryable field, which could later be populated, correlated against access logs, or leaked through an incomplete redaction in a future code change. Omission removes the field from the document schema instance entirely, which is the stronger guarantee.

C. Threat Model and Scope

The anonymity property in this paper is stated, deliberately, as a data-layer guarantee, not a network- or infrastructure-layer guarantee. Three boundaries of the threat model follow directly from this scope:

Boundary 1 — Application-layer adversary. The property protects against an adversary who can query the MongoDB database or call the authenticated REST API (including a malicious or compromised administrator account), since neither path can recover u for an anonymous document because the value is not stored anywhere in the system.

Boundary 2 — Infrastructure-layer adversary excluded. The property does **not** by itself protect against an adversary with access to infrastructure outside the MongoDB document — for example, Express access logs, Vercel deployment logs, or TLS termination logs at the hosting provider, which may record the source IP address or session metadata of the HTTP request independently of anything written to MongoDB. ACMS's anonymity claim, precisely stated, is: **the application's own persistent data store never records who submitted an anonymous complaint.** It is not a claim that no system anywhere in the deployment stack retains any trace of the request.

Boundary 3 — Single point of enforcement. The guarantee depends entirely on the correctness of the single conditional in Section V-B executing on every write path that creates a Complaint document. There is currently one

controller and one creation path, which keeps this assumption easy to audit, but it is an assumption rather than a structural impossibility enforced by, for instance, a database-level constraint independent of application code.

D. Relationship to Access Control

Anonymity (Section VI-B) and access control (role-based authorization in Section V-C) are independent properties that compose rather than substitute for one another. Role middleware determines **who may read** a complaint document; the anonymity property determines **what is in** the document to be read. A flaw in role middleware that exposed the Complaint collection to an unauthorized party would still not reveal a submitter for an anonymous complaint, because that information is not present in the leaked data — the two failure modes are independent. This compositional separation is a design strength relative to systems (Section II-B) that rely solely on access-control policy to provide what is described as anonymity.

VII. IMPLEMENTATION

A. Technology Stack

Table IV: Technology Stack and Justification

Layer	Technology	Justification
UI library	React.js	Component reuse; virtual DOM; large ecosystem
Styling	Tailwind CSS	Utility-first; responsive by default
Routing	React Router	Role-aware client-side routing
HTTP client	Axios	Interceptors for automatic JWT header injection
Server runtime	Node.js	Non-blocking I/O; JavaScript across the stack
API framework	Express.js	Minimal, well-documented REST routing
Authentication	JWT (jsonwebtoken)	Stateless, signed tokens issued by the Express server
Password security	bcrypt	Adaptive hashing; brute-force resistant
Database	MongoDB	Flexible document model for variable complaint shapes
ODM	Mongoose	Schema validation and references on top of MongoDB
Database hosting	MongoDB Atlas	Managed cloud cluster for production
Local development	MongoDB Compass	Direct collection inspection during testing
Frontend deployment	Vercel	Static hosting, global CDN, automatic HTTPS
Version control	GitHub	github.com/vitthalkendre29/complaintmanagementsystem

B. Core Anonymity Mechanism

The anonymity mechanism is implemented in the Express controller for complaint creation. When the client sends isAnonymous: true, the controller does not attach a submittedBy field to the document passed to Mongoose, so MongoDB never stores a reference to the submitting student:

```
// controllers/complaintController.js
exports.createComplaint = async (req, res) =>
{
  const { title, description, category,
    priority, isAnonymous } = req.body;

  // CORE ANONYMITY: only attach the
  // submitter reference if not anonymous
  const complaintData = {
    title, description, category,
    priority, status: 'Open'
  };

  if (!isAnonymous) {
    complaintData.submittedBy = req.user.id;
  }

  const complaint = await Complaint.create(
    complaintData
  );

  res.status(201).json({
    id: complaint._id,
    status: complaint.status
  });
};
```

C. JWT Authentication and Role Middleware

Two Express middleware functions enforce authentication and role-based access control on all protected API routes. verifyToken validates the JWT on the request header; requireRole checks the decoded role against the route's required role:

```
// middleware/auth.js
const jwt = require('jsonwebtoken');

exports.verifyToken = (req, res, next) => {
  const header = req.headers.authorization;
  const token = header &&
    header.split(' ')[1];
  if (!token)
    return res.status(401).json(
      { error: 'No token provided' });

  try {
    req.user = jwt.verify(
      token, process.env.JWT_SECRET);
    next();
  } catch {
    res.status(401).json(
      { error: 'Invalid or expired token' });
  }
};

exports.requireRole = (role) =>
  (req, res, next) => {
    if (req.user.role !== role)
      return res.status(403).json(
        { error: 'Access forbidden' });
    next();
  };
```

D. Password Security with bcrypt

All passwords are hashed using bcrypt before being saved to MongoDB. Plaintext passwords are never stored or logged at any point in the system:

```
// controllers/authController.js
const bcrypt = require('bcrypt');
const SALT_ROUNDS = 10;

// Registration
const hashedPassword = await bcrypt.hash(
  req.body.password, SALT_ROUNDS
);
const user = await User.create({
  ...req.body, password: hashedPassword
});

// Login
const isValid = await bcrypt.compare(
  req.body.password, user.password
);
if (!isValid)
  return res.status(401).json(
    { error: 'Invalid credentials' });

const token = jwt.sign(
  { id: user._id, role: user.role },
  process.env.JWT_SECRET,
  { expiresIn: '24h' }
);
```

E. Mongoose Schema Definition

The Complaint schema is defined with Mongoose, leaving submittedBy as an optional reference field rather than a required one, which is what allows the anonymity mechanism in Section V-B and the formal property in Section VI-B to function correctly:

```
// models/Complaint.js
const complaintSchema = new Schema({
  title: { type: String, required: true },
  description: { type: String,
    required: true },
  category: { type: String,
    required: true },
  priority: { type: String,
    enum: ['Low','Medium','High','Critical'],
    default: 'Low' },
  status: { type: String,
    enum: ['Open','Pending','In Progress',
      'Resolved','Rejected'],
    default: 'Open' },
  submittedBy: { type:
Schema.Types.ObjectId,
    ref: 'User' } // not required
}, { timestamps: true });
```

F. Deployment

The React frontend is built with npm run build and deployed on **Vercel**, serving the static bundle through a global CDN with automatic HTTPS. The Express.js API is deployed separately as a Node.js service, connecting to a MongoDB Atlas cluster via a connection string stored in an environment variable. Local development and debugging used MongoDB Compass to inspect collections directly against a local or Atlas-connected database. The live frontend is accessible at *complaintmanagementsystem-eight.vercel.app*, with the frontend source published at *github.com/vitthalkendre29/complaintmanagementsystem*.

VIII. TESTING AND FUNCTIONAL EVALUATION

A. Testing Strategy

Functional testing was conducted manually against the deployed system using Chrome (desktop, 1920×1080) and Chrome Mobile DevTools (375px viewport), exercising both the student and administrator dashboards end to end, with API requests additionally verified using Postman.

B. Functional Test Results

Table V: Functional Test Cases and Results

TC-ID	Test Case	Expected Output	Result
TC-01	Student registration	HTTP 201; password stored as bcrypt hash	**PASS**
TC-02	Valid login	HTTP 200; signed JWT returned with role claim	**PASS**
TC-03	Invalid login	HTTP 401; no token issued	**PASS**
TC-04	Anonymous submission	submittedBy absent in MongoDB document; admin sees “Anonymous”	**PASS**
TC-05	Named submission	submittedBy = current user's ObjectId	**PASS**
TC-06	Student calls admin-only endpoint	HTTP 403 from role middleware	**PASS**
TC-07	Request without JWT	HTTP 401 from verifyToken middleware	**PASS**
TC-08	Admin status update	Status change visible immediately in student view	**PASS**
TC-09	Admin reply to student	Reply visible in student's complaint detail	**PASS**
TC-10	Complaint filter by category/status	Only matching complaints displayed	**PASS**
TC-11	Responsive layout at 375px mobile	All UI elements visible; forms functional	**PASS**

C. Security Validation

Inspection of the MongoDB Atlas collection via Compass confirmed that all stored passwords are bcrypt hashes; no plaintext password was found in any document. Expired or malformed JWTs were consistently rejected with HTTP 401 by the verifyToken middleware. Requests from a valid student token to admin-only endpoints consistently returned HTTP 403 from requireRole. Anonymous complaint documents were verified directly in Compass to have no submittedBy field present at all, rather than a null or placeholder value, confirming the anonymity mechanism operates as designed at the data layer.

D. Discussion of the Custom MERN Backend

Building a custom Express.js API, rather than adopting a managed Backend-as-a-Service, gave the project team full control over validation logic, the exact shape of JWT claims, and the precise conditions under which the submittedBy field is populated — control that is harder to express declaratively in a platform-managed access-control layer. The trade-off is that the team is responsible for deploying, securing, and maintaining the API server itself, including correctly configuring CORS, environment variables, and the MongoDB Atlas network access list. For a four-person team with prior coursework experience in Express and MongoDB, this was a manageable trade-off and kept the entire authentication and anonymity logic auditable in one codebase.

IX. RESULTS AND DISCUSSION

This section synthesises the outcomes of Sections VI through VIII into a discussion of what was actually demonstrated, evaluated qualitatively against the goals stated in Section I rather than against synthetic performance benchmarks that were not run on the live deployment.

A. Summary of Demonstrated Properties

Three properties were verified directly against the deployed system rather than assumed from the design:

- **Genuine field-level anonymity** — confirmed in MongoDB Compass (Section VIII-C) by the literal absence of the submittedBy key on anonymous documents, matching the formal property stated in Section VI-B, rather than a null or masked value that could later be reversed.
- **Consistent access control** — every one of the eleven functional test cases involving an authentication or authorization boundary (TC-02, TC-03, TC-06, TC-07 in Table V) produced the expected HTTP status code, with no observed case of a student-scoped request reaching an admin-only handler.
- **Password security at rest** — direct inspection of the Atlas collection confirmed bcrypt hashes only, with no plaintext password recoverable from the data store.

B. Comparison Against Stated Goals

Section I framed the core contribution as a data-model-level anonymity mechanism, in contrast to the policy-only anonymity offered by the systems surveyed in Section II. Table II's existing-system column is drawn from that survey rather than from a single named competitor, and ACMS's omitted-reference mechanism is, on the available evidence, a stronger technical guarantee than the access-control-only anonymity reported for the institutional grievance portals in Kumar and Singh [3]. This comparison is necessarily qualitative: the related work does not publish data-layer verification of its own anonymity claims, so a direct quantitative comparison is not possible from public information, and this paper does not attempt to manufacture one.

C. Discussion of Design Trade-offs

The decision to build a custom Express.js backend rather than adopt a managed Backend-as-a-Service (discussed first in Section III-B and revisited in Section VIII-D) is, in retrospect, the trade-off with the largest effect on what this paper is able to claim. Because the team controls the exact write path into MongoDB, the anonymity property in Section VI could be verified at the level of individual controller logic and individual stored documents. A platform-managed backend would have made the same claim harder to verify independently of the platform vendor's own access-control configuration, even if the net behaviour were similar.

The cost of this choice is operational: the team is responsible for correctly configuring CORS, environment variables, and the JWT secret, and a misconfiguration in any of these is a code-level risk rather than something a managed platform would prevent by default. Section VIII-D's discussion of this trade-off should be read together with the limitations in Section X, particularly the absence of an external security audit.

X. LIMITATIONS

This section states the limitations of the present work explicitly, distinguishing limitations of scope (what the system and this paper do not attempt) from limitations of evidence (claims that could be stronger with additional verification not yet performed).

A. Scope Limitations

1. **No protection against infrastructure-layer correlation.** As stated formally in Section VI-C (Boundary 2), the anonymity guarantee covers the application's own MongoDB data store. It does not extend to network-level metadata such as source IP addresses that may be retained in hosting-provider or CDN logs outside the application's control, and this paper makes no claim about those layers.
2. **No formal verification of the implementation.** The anonymity property in Section VI is demonstrated by code inspection and direct data inspection (Section VIII-C), not by machine-checked formal verification of the Express controller. A change to the controller that reintroduced the submittedBy field under some code path would not be caught automatically; it would require the same manual inspection process repeated.
3. **Single-institution, single-deployment evaluation.** All functional and security verification in Section VIII was performed against one deployment serving one institution's workflow. The findings have not been replicated across multiple institutions or multiple independent deployments of the codebase.

B. Evidentiary Limitations

1. **No load or performance benchmarking was conducted.** This paper deliberately does not report request latency, throughput, or concurrent-user capacity figures. The authors' sandboxed evaluation environment for this manuscript did not have network access to the production Vercel and MongoDB Atlas endpoints, and rather than substitute illustrative or externally sourced figures that were not measured on this system, this paper omits performance claims entirely. A controlled load-testing study (e.g., using a tool such as k6 or Apache JMeter against a staging deployment) is identified as future work in Section XI rather than reported here.
2. **No independent security audit.** Section VII-F notes that a formal review of JWT secret rotation and CORS configuration has not yet been performed. The security validation in Section VIII-C covers the specific properties tested (password hashing, token rejection, role enforcement, field omission) and should not be read as a general security audit of the deployed system.
3. **Functional testing was manual and author-conducted.** The eleven test cases in Table V were executed manually by the project team rather than by an independent party or an automated test suite with code-coverage reporting. This is appropriate evidence for the specific behavioural claims made in this paper but is a weaker standard of evidence than an independently audited or automated regression suite would provide.
4. **No user study.** No data was collected from actual students or administrators on perceived trust in the anonymity mechanism, usability of the dashboards, or willingness to file a complaint that would not otherwise have been filed. The motivation in Section I (fear of identification deterring legitimate complaints) is supported by the cited literature [2], [3] rather than by new survey data collected as part of this work.

C. Why These Limitations Are Stated Explicitly

A paper documenting an anonymity mechanism has a particular obligation to be precise about the

boundary of what is actually guaranteed, since overstating an anonymity or security property is more harmful than overstating an ordinary functional feature: a reader — in this case potentially a student deciding whether to trust the system with a sensitive complaint — may rely on the claim under conditions the authors did not test. The limitations above are accordingly stated as part of the contribution of this paper, not as an afterthought.

XI. CONCLUSION AND FUTURE WORK

This paper presented the Anonymous Complaint Management System (ACMS) as implemented on the MERN stack: a React.js single-page application calling a custom Node.js and Express.js REST API, with MongoDB as the data store accessed through Mongoose. Anonymity is enforced by omitting the submitter reference field from a complaint document when a student opts to submit anonymously, a mechanism formalised in Section VI and verified directly against stored data in Section VIII-C, while access control is enforced through JWT verification and role-checking middleware on every protected route.

All eleven functional test cases passed, including direct verification in MongoDB Compass that anonymous complaint documents contain no submitter reference whatsoever. The system is currently live and operational, with the frontend source code publicly available for inspection. Section X states the scope and evidentiary limitations of this evaluation explicitly, including the absence of load testing and independent security audit, so that the claims above are read with their proper boundaries rather than as a general security guarantee.

Future Work: (i) **Controlled performance evaluation** of the deployed API under synthetic load, addressing the benchmarking gap identified in Section X-B; (ii) **AI-based auto-classification** of complaint category and priority using a lightweight model called from the Express API; (iii) **Email notifications** via Nodemailer triggered from the Express controller on status change; (iv) **A Flutter mobile client** consuming the same REST API; (v) **Audit logging** of all status transitions in a dedicated MongoDB collection for institutional compliance reporting; (vi) **Two-Factor Authentication** for administrator accounts using TOTP verified in the Express layer; (vii) an **independent security audit** of JWT secret rotation and CORS configuration, and (viii) a **small-scale user study** with students and administrators to evaluate perceived trust and usability, before any wider institutional rollout.

ACKNOWLEDGMENT

The authors express sincere gratitude to Prof. Mrs. R. T. Waghmode, Department of Computer Engineering, Sinhgad Institute of Technology and Science, Narhe, Pune, for her continuous guidance, constructive feedback, and constant encouragement throughout this project. The authors also thank the Department of Computer Engineering and SITS, affiliated to Savitribai Phule Pune University, for providing the necessary infrastructure and support during the academic year 2025–26.

REFERENCES

[1] H. Suryotrisongko, D. P. Jayanto, and A. Tjahyanto, "Design and Development of Backend Application for Public Complaint Systems Using Microservice Spring Boot," Procedia Computer Science, vol. 124, pp. 736–743, 2017, doi: 10.1016/j.procs.2017.12.212. Published by Elsevier B.V., 4th Information Systems International Conference (ISICO 2017), Bali, Indonesia.

[2] A. Joshi, R. Patel, and M. Sharma, "Participation barriers in e-governance grievance portals: Identity disclosure and fear of retaliation," International Journal of E-Government Research, vol. 17, no. 3, pp. 45–62, 2021.

[3] R. Kumar and T. Singh, "Evaluation of online complaint management systems in Indian universities," Journal of Educational Technology & Society, vol. 23, no. 2, pp. 112–128, 2020.

[4] NAVEX Global, EthicsPoint Hotline and Incident Management Platform,

https://www.navex.com, Accessed: Feb. 10, 2025.

[5] C. Date, An Introduction to Database Systems, 8th ed. Boston, MA: Pearson Addison-Wesley, 2003.

[6] K. Mivule, "Utilizing noise addition for data privacy, an overview," Proc. International Conference on Information and Knowledge Engineering, pp. 65–71, 2012.

[7] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF, RFC 7519, May 2015. https://tools.ietf.org/html/rfc7519

[8] N. Provos and D. Mazières, "A future-adaptable password scheme," Proc. USENIX Annual Technical Conference (FREENIX Track), pp. 81–91, 1999.

[9] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed NIST standard for role-based access control," ACM Transactions on Information and System Security, vol. 4, no. 3, pp. 224–274, Aug. 2001.

[10] Meta Open Source, React – A JavaScript Library for Building User Interfaces, https://react.dev, Accessed: Jan. 15, 2025.

[11] MongoDB Inc., Mongoose Documentation, https://mongoosejs.com/docs/guide.html, Accessed: Jan. 2025.